

```

import numpy as np
from model import *
from flask import Flask, request, render_template

# Create flask app
flask_app = Flask(__name__)

@flask_app.route("/")
def Home():
    return render_template("home.html")

@flask_app.route("/predict", methods=['GET', 'POST'])
def predict():
    if request.method == 'POST':
        pitcher_name = request.form['pitcher_name']
        batter_name = request.form['batter_name']
        balls_encoded = int(request.form['balls_encoded'])
        strikes_encoded = int(request.form['strikes_encoded'])
        inning_encoded = int(request.form['inning_encoded'])
        outs_encoded = int(request.form['outs_encoded'])
        on_3b_encoded = int(request.form['on_3b_encoded'])
        on_2b_encoded = int(request.form['on_2b_encoded'])
        on_1b_encoded = int(request.form['on_1b_encoded'])
        predictions, best_zone, best_pitch_type, possible_combinations,
pitch_type_associated = main(pitcher_name, batter_name, balls_encoded,
strikes_encoded, outs_encoded, on_3b_encoded, on_2b_encoded, on_1b_encoded,
inning_encoded)

        print(f"The best zone for the pitcher is: {best_zone}")
        print(f"The best pitch type for the pitcher is: {best_pitch_type}")

        predictions = pd.DataFrame(predictions)

        # Calculate the total number of groups
        total_groups = len(predictions) // 13

        # Initialize an empty list to store the values for the heatmap
        heatmap_data_list = []

        # Loop through the indices and extract the corresponding values
        for value_idx in range(best_pitch_type, 9 * total_groups, total_groups):
            group_values = predictions.iloc[value_idx, 0] # Get the value at the
specified index
            heatmap_data_list.append(group_values)

        # Reshape the values into a 3x3 grid
        heatmap_data = pd.Series(heatmap_data_list).values.reshape(3, 3)
        print(heatmap_data)
        # Create the heatmap plot
        plt.figure(figsize=(6, 6))

```

```

plt.imshow(heatmap_data, cmap='bwr', interpolation='gaussian',
vmin=predictions.min(), vmax=predictions.max())

# Loop to display numbers in the middle of each zone
for i in range(3):
    for j in range(3):
        plt.text(j, i, str(i*3 + j + 1), ha='center', va='center',
color='white')

# Remove x and y tick marks
plt.xticks([])
plt.yticks([])

plt.colorbar()
plt.title(f"The best pitch type and zone is:\n {best_pitch_type}
{pitch_type_associated} in zone: {best_zone}")
plt.savefig('static/my_heatmap_plot.png')
else:
    return render_template('predict.html')

return render_template("predict.html", prediction_message = f"Best Zone:
{best_zone}, Best Pitch Type: {best_pitch_type} ", heatmap_url =
"static/my_heatmap_plot.png", plot_url ="static/my_prediction_plot.png")

@flask_app.route('/about/')
def about():
    return render_template('about.html')

@flask_app.route("/projects/", methods=['GET','POST'])
def Projects():
    if request.method == 'POST':
        pitcher_id = request.form['pitcher_id']
        scaledcluster = spin_data(pitcher_id)

inertias = []
for i in range(1,8):
    kmeans = KMeans(n_clusters=i)
    kmeans.fit(scaledcluster)
    inertias.append(kmeans.inertia_)

kmeans = KMeans(n_clusters=3)
kmeans.fit(scaledcluster)

```

```

scaledcluster['kmeans_3'] = kmeans.labels_

tsne = TSNE(n_components=2, verbose=1, perplexity=30, n_iter=1000)
scaledcluster.columns = scaledcluster.columns.astype(str)
proj = tsne.fit_transform(scaledcluster)
tsneresults = pd.DataFrame(proj, columns=['tsne1', 'tsne2'])
tsneresults.columns = tsneresults.columns.astype(str)
plt.scatter(x="tsne1", y="tsne2", c=scaledcluster['kmeans_3'],
data=tsneresults)
plt.title('TSNE Clustering with 3 dimensions')
plt.savefig('static/my_cluster_plot.png')
else:
    return render_template('projects.html')

return render_template("projects.html", cluster_url =
"static/my_cluster_plot.png", elbow_url = "static/my_elbow_plot.png" )

if __name__ == "__main__":
    flask_app.run(debug=False)

```